

Solidum FEM

Plataforma de Análisis por Elementos Finitos en Python

Sigla: FF-ME

Manual de Ejemplos

Problemas resueltos y verificados contra
soluciones analíticas y de referencia

Autor: Jaime Retama Velasco

Facultad de Estudios Superiores Aragón
Universidad Nacional Autónoma de México

23 de septiembre de 2026

Índice general

Sobre este manual	III
1. Voladizo con carga en el extremo: marcos de Euler-Bernoulli y de Timoshenko	1
1.1. Planteamiento	2
1.2. Solución analítica	2
1.3. Modelo en Solidum	3
1.4. Resultados	4
1.5. Deformación por cortante: Timoshenko frente a Euler-Bernoulli	5
1.6. Para seguir	7
2. Placa con agujero a tracción: concentración de esfuerzos de Kirsch	9
2.1. Planteamiento	9
2.2. Solución analítica	10
2.3. Modelo en Solidum	10
2.4. Resultados	12
2.5. Dos fuentes de error	14
2.6. Para seguir	14
3. Cilindro de pared gruesa: estudio de convergencia con el API de Python	16
3.1. Planteamiento	16
3.2. Solución analítica	17
3.3. Modelo con el API de Python	17
3.4. Estudio de convergencia	19
3.5. Lineales frente a cuadráticos	21
3.6. Para seguir	21
4. Cilindro elastoplástico: primera fluencia, colapso y bloqueo volumétrico	23
4.1. Planteamiento	23
4.2. Soluciones analíticas	23
4.3. Modelo en Solidum	24
4.4. Resultados	26
4.5. Bloqueo volumétrico	27
4.6. Para seguir	27

5. Barra con daño: localización, retroceso y control indirecto	28
5.1. Planteamiento	28
5.2. Solución analítica	28
5.3. Modelo en Solidum	29
5.4. Resultados	31
5.5. Para seguir	33

Sobre este manual

Este manual reúne **problemas resueltos** con Solidum FEM. Cada capítulo plantea un problema de mecánica de sólidos, lo modela paso a paso —con el archivo `.yaml` o con el API de Python—, compara los resultados con una solución analítica o de referencia publicada y comenta qué enseña el ejemplo sobre el método o sobre el programa.

Ninguna cifra ni figura está escrita a mano. Cada ejemplo tiene una carpeta en `examples/` con su modelo y un script `run.py` que resuelve el problema, guarda los resultados en `resultados.json` y dibuja las figuras; el manual los lee de ahí al compilarse. Un test de la suite (`tests/test_examples_manual.py`) ejecuta los mismos scripts y comprueba que los errores citados siguen dentro de su tolerancia: lo que este manual afirma se verifica en cada cambio del código.

Para reproducir un ejemplo basta con

```
python examples/carpeta/run.py
```

Para la sintaxis completa del archivo de entrada, consultar el *Manual de Usuario* (`manuals/User_manual.pdf`); para la formulación de cada elemento, material y solver, el *Manual de Referencia* (`manuals/Reference_manual.pdf`).

Este manual se regenera con:

```
python manuals/build_example_manual.py
```

No editar este PDF manualmente; cualquier corrección debe hacerse sobre los capítulos en `manuals/sources/examples/` o sobre los scripts de `examples/`.

Capítulo 1

Voladizo con carga en el extremo: marcos de Euler-Bernoulli y de Timoshenko

El voladizo con carga puntual en el extremo es el problema más sencillo con solución cerrada, y por eso es el primero. Sirve para tres cosas: comprobar que el modelo reproduce la teoría de vigas a precisión de máquina, aprender a leer los resultados (desplazamientos, reacciones y fuerzas internas) con la convención de signos de Solidum, y medir cuánto aporta la deformación por cortante según la esbeltez.

Qué se aprende

- Escribir un modelo YAML de marco 2D: nodos, material, elementos, apoyos y cargas.
- Leer desplazamientos, giros, reacciones y fuerzas internas N , V , M del resultado.
- Interpretar el signo de las fuerzas internas 2D según la convención de viga del programa.
- Por qué la interpolación hermítica cúbica da la solución exacta con cargas nodales.
- Cuándo importa la deformación por cortante, y por qué el elemento de Timoshenko de Solidum no sufre bloqueo por cortante.

1.1 Planteamiento

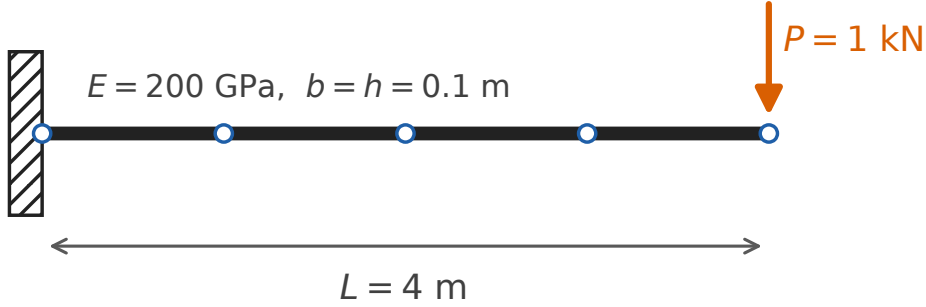


Figura 1.1: Voladizo empotrado en $x=0$ con carga puntual P en el extremo libre. Los círculos marcan los cinco nodos del modelo.

La viga de [figura 1.1](#) tiene longitud $L=4 \text{ m}$, está empotrada en $x=0$ y libre en $x=L$. La sección es cuadrada, $b=h=0.1 \text{ m}$, con área $A=0.01 \text{ m}^2$ y momento de inercia $I=bh^3/12=8.3 \times 10^{-6} \text{ m}^4$. El material es acero, $E=200 \text{ GPa}$. En el extremo libre actúa una carga $P=1 \text{ kN}$ hacia abajo, es decir, en $-y$. Todas las magnitudes se dan en unidades SI coherentes: m, N y Pa.

1.2 Solución analítica

Con $+y$ hacia arriba y giro antihorario positivo, la teoría de Euler-Bernoulli da la deformada y el giro

$$u_y(x) = -\frac{P x^2 (3L - x)}{6EI}, \quad \theta(x) = \frac{du_y}{dx} = -\frac{P x (2L - x)}{2EI},$$

de modo que en el extremo libre la flecha es $\delta = PL^3/(3EI)$ y el giro $\theta_L = -PL^2/(2EI)$, negativo porque la sección gira en sentido horario.

Las fuerzas internas se obtienen cortando la viga en x y aislando el tramo derecho. En la convención de viga del programa, la cara izquierda de ese tramo (normal saliente $-x$) lleva V positivo hacia $+y$ y M positivo en sentido horario. El equilibrio vertical del tramo da $V - P = 0$, y el de momentos respecto al corte da $M = -P(L - x)$:

$$N(x) = 0, \quad V(x) = +P, \quad M(x) = -P(L - x).$$

El cortante es positivo porque tiende a girar el diferencial en sentido horario. El flector es negativo en toda la viga, lo que indica tracción en la fibra superior, como corresponde a un voladizo cargado hacia abajo. El empotramiento reacciona con $R_y = +P$ hacia arriba y con un momento $M_z = +PL$ antihorario.

1.3 Modelo en Solidum

El modelo completo es el archivo `examples/voladizo_marco/modelo.yaml`:

```
# SOLIDUM FEM - Manual de ejemplos, ejemplo 1
# Voladizo con carga puntual en el extremo libre (marco 2D, Euler-Bernoulli)
# Unidades: m, N, Pa
#
# L = 4 m, sección cuadrada 0.1 m × 0.1 m, acero E = 200 GPa, P = 1 kN hacia abajo.
# Cuatro elementos de 1 m: con carga sólo en los nodos, la interpolación
# hermítica cúbica es la solución exacta de la ecuación de la viga, así que
# los valores nodales deben coincidir con la solución analítica a precisión
# de máquina.

nodes:
  - {id: 1, coords: [0.0, 0.0]}
  - {id: 2, coords: [1.0, 0.0]}
  - {id: 3, coords: [2.0, 0.0]}
  - {id: 4, coords: [3.0, 0.0]}
  - {id: 5, coords: [4.0, 0.0]}

materials:
  - {id: 1, type: Elastic1D, E: 200.0e9}

# A = b·h, I = b·h3/12 (m2, 4m)
elements:
  - {id: 1, type: Frame2DEuler, material: 1, nodes: [1, 2],
    A: 0.01, I: 8.333333333333e-6}
  - {id: 2, type: Frame2DEuler, material: 1, nodes: [2, 3],
    A: 0.01, I: 8.333333333333e-6}
  - {id: 3, type: Frame2DEuler, material: 1, nodes: [3, 4],
    A: 0.01, I: 8.333333333333e-6}
  - {id: 4, type: Frame2DEuler, material: 1, nodes: [4, 5],
    A: 0.01, I: 8.333333333333e-6}

# Empotramiento: desplazamientos y giro impedidos en x = 0
boundary_conditions_by_node:
  - {node_id: 1, ux: 0.0, uy: 0.0, rz: 0.0}

# P = 1 kN hacia abajo -(y) en el extremo libre
point_loads_by_node:
  - {node_id: 5, uy: -1000.0}

solver:
  type: LinearSolver
```

Los puntos que conviene notar:

- **Nodos y grados de libertad.** Cada nodo de un elemento `Frame2DEuler` tiene tres grados de libertad: `ux`, `uy` y `rz`. Los declara el elemento; el YAML sólo da coordenadas.
- **Material.** `Elastic1D` sólo necesita E . El área y la inercia son propiedades de la sección y van en cada elemento.
- **Empotramiento.** Fijar `ux`, `uy` y `rz` del nodo 1 impide los dos desplazamientos y el giro.

- **Carga.** `uy`: `-1000.0` en el nodo 5 es una fuerza de 1 kN en $-y$. Las cargas nodales se dan en ejes globales.
- **Solver.** `LinearSolver` resuelve $\mathbf{K}\mathbf{u} = \mathbf{F}$ una sola vez. Antes de factorizar comprueba que el modelo no sea un mecanismo, y después verifica el equilibrio de la solución.

El script `run.py` resuelve este YAML con el mismo proceso que `solidum.run_yaml`, pero conserva el dominio para poder indexar el vector de desplazamientos por nodo (`node.dofs["uy"]`). Las reacciones y las fuerzas internas vienen en el resultado: `res.reactions_by_node[1]` es un diccionario `{"ux": ..., "uy": ..., "rz": ...}` y `res.element_forces[e].components["M"]` contiene el flector en los dos extremos del elemento `e`, en sus ejes locales.

1.4 Resultados

Tabla 1.1: Voladizo de Euler-Bernoulli: Solidum frente a la solución analítica.

Magnitud	Solidum	Analítica
Flecha en el extremo δ [mm]	12.800000	12.800000
Giro en el extremo θ_L [rad]	-4.800000×10^{-3}	-4.800000×10^{-3}
Reacción R_y [N]	1000.000000	1000.000000
Momento de reacción M_z [N·m]	4000.000000	$PL = 4000$
Flector en el empotramiento $M(0)$ [N·m]	-4000.000000	$-PL = -4000$
Cortante V [N]	1000.000000	1000.000000

La mayor discrepancia relativa entre todas las magnitudes comparadas, incluidos los desplazamientos de los cinco nodos y las fuerzas internas en los dos extremos de cada elemento, es menor que 10^{-13} : el redondeo de la aritmética de doble precisión.

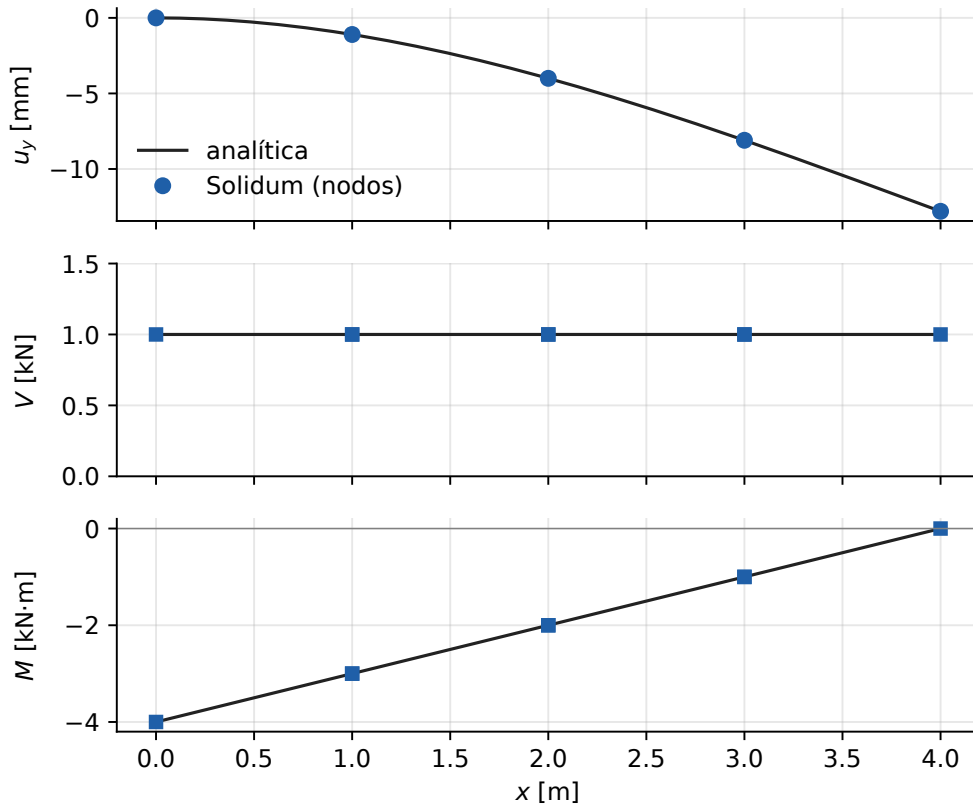


Figura 1.2: Deformada, cortante y flector del voladizo. Línea: solución analítica. Marcadores: Solidum, en los nodos (deformada) y en los dos extremos de cada elemento (fuerzas internas).

La [figura 1.2](#) muestra que la coincidencia no se limita al extremo. El resultado exacto tiene una explicación precisa: sin carga entre nodos, la ecuación de la viga $EI u_y'''' = 0$ tiene por solución un polinomio cúbico en cada tramo, y las funciones de forma hermíticas del elemento son justamente cúbicas. El método de elementos finitos reproduce entonces la solución exacta en los nodos, con cualquier número de elementos; los cuatro de este modelo sólo sirven para dibujar la deformada. Con una carga distribuida la solución exacta pasa a ser de cuarto grado y deja de estar contenida en el espacio de interpolación: los valores nodales siguen siendo exactos en el caso de la viga, pero la deformada entre nodos ya no.

Los signos de la tabla confirman la convención: el cortante sale positivo, el flector negativo (tracción arriba) y la reacción de momento positiva (antihoraria).

1.5 Deformación por cortante: Timoshenko frente a Euler-Bernoulli

La teoría de Euler-Bernoulli supone que las secciones planas permanecen perpendiculares al eje deformado y desprecia la deformación por cortante. La de Timoshenko la incluye, y la flecha en el extremo del voladizo gana un término:

$$\delta_T = \frac{PL^3}{3EI} + \frac{PL}{\kappa GA}, \quad \frac{\delta_T}{\delta_{EB}} = 1 + \frac{3EI}{\kappa GAL^2},$$

donde κA es el área efectiva de cortante (parámetro A_s del elemento). Para una sección rectangular, con $I/A = h^2/12$, $G = E/[2(1 + \nu)]$ y $\kappa = 5/6$, el cociente depende sólo de la esbeltez:

$$\frac{\delta_T}{\delta_{EB}} = 1 + \frac{1 + \nu}{2\kappa} \left(\frac{h}{L} \right)^2 = 1 + 0.78 \left(\frac{h}{L} \right)^2 \quad (\nu = 0.3).$$

El script repite el voladizo con una sección de 0.1×0.2 m y esbelteces L/h de 1 a 1000, cada vez con **un solo elemento** `Frame2DTimoshenko` y un solo `Frame2DEuler`. Este ejemplo usa el API de Python en vez del YAML: el modelo se construye nodo a nodo.

```
def _flecha_punta(cls, L_h: float) -> float:
    """Flecha en la punta de un voladizo de un solo elemento, sección b × h = 0.1 × 0.2 m."""
    h = 0.2
    b = 0.1
    Lb = L_h * h
    A = b * h
    Ib = b * h**3 / 12.0
    domain = Domain()
    mat = Elastic1D(E=E)
    n1 = domain.add_node(1, [0.0, 0.0])
    n2 = domain.add_node(2, [Lb, 0.0])
    if cls is Frame2DTimoshenko:
        el = Frame2DTimoshenko(1, [n1, n2], mat, A=A, I=Ib, As=KAPPA * A, nu=NU)
    else:
        el = Frame2DEuler(1, [n1, n2], mat, A=A, I=Ib)
    domain.add_element(el)
    for d in ("ux", "uy", "rz"):
        n1.fix_dof(d, 0.0)
    domain.generate_equation_numbers()
    F = np.zeros(domain.total_dofs)
    F[n2.dofs["uy"]] = -P
    U = LinearSolver(Assembler(domain)).solve(F)
    return -U[n2.dofs["uy"]]
```

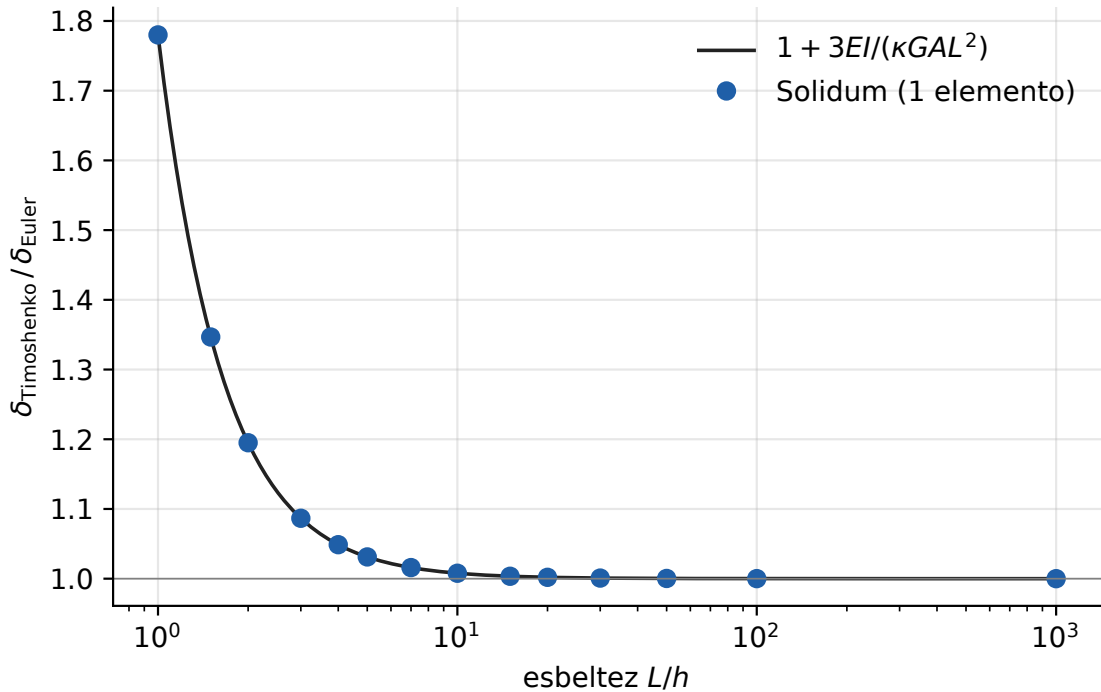


Figura 1.3: Cociente de flechas Timoshenko / Euler-Bernoulli en función de la esbeltez. Cada punto de Solidum es un modelo de un solo elemento.

La figura 1.3 compara el cociente calculado con la expresión analítica. La discrepancia máxima es menor que 10^{-14} . Dos lecturas prácticas:

- **Cuándo importa el cortante.** En $L/h = 10$ el cortante añade un 0.78 % a la flecha; en $L/h = 5$, un 3.12 %; en una viga tan corta como peraltada, $L/h = 1$, la flecha es 1.78 veces la de Euler-Bernoulli. La regla habitual de usar Timoshenko por debajo de $L/h \approx 10$ sale directamente de esta curva.
- **Sin bloqueo por cortante.** En $L/h = 1000$ el cociente excede la unidad en 7.8×10^{-7} , como predice la teoría. Un elemento de Timoshenko con interpolación lineal independiente de flecha y giro, integrado de forma completa, se bloquea en ese límite: su rigidez a cortante domina y la flecha tiende a cero en vez de tender a la de Euler-Bernoulli. Frame2DTimoshenko usa la rigidez exacta de la viga de Timoshenko, escrita con el factor $\Phi = 12EI/(\kappa GA L^2)$; equivale a funciones de forma que dependen de Φ y se reducen a las hermíticas cuando $\Phi \rightarrow 0$. Por eso un elemento basta a cualquier esbeltez.

1.6 Para seguir

- Cambiar un elemento del YAML por `Frame2DTimoshenko`, con `As: 0.008333333` y `nu: 0.3`. Como el cortante es constante, la flecha en el extremo debe crecer en $P\ell/(\kappa GA)$, con $\ell = 1$ m la longitud del tramo cambiado.

- Añadir peso propio con el bloque `gravity` y la `density` del material. La carga distribuida hace que el flector entre nodos sea parabólico; los valores de extremo de `element_forces` descuentan la carga nodal equivalente y siguen siendo exactos.
- El mismo voladizo modelado con sólidos 2D de cuatro nodos muestra el bloqueo por cortante que aquí no aparece. Lo documenta el benchmark de MacNeal-Harder en `tests/validation/test_slender_cantilever.py`.

Archivos del ejemplo. La carpeta `examples/voladizo_marco/` contiene el modelo `modelo.yaml`, el script `run.py`, los resultados en `resultados.json` y las figuras. Para regenerar resultados y figuras basta con ejecutar el script.

Capítulo 2

Placa con agujero a tracción: concentración de esfuerzos de Kirsch

Un agujero circular en una placa traccionada multiplica por tres el esfuerzo en su borde. Es el caso clásico de concentración de esfuerzos, resuelto por Kirsch en 1898 para una placa infinita, y el primer problema bidimensional de este manual. A diferencia del voladizo, aquí la solución de elementos finitos ya no es exacta: el ejemplo enseña a medir cuánto se aparta de la analítica, dónde y por qué.

Qué se aprende

- Resolver una malla de Gmsh desde el YAML y aplicar condiciones de contorno por grupo físico.
- Aprovechar la doble simetría para modelar sólo un cuarto de la placa.
- Cargar con desplazamiento impuesto y obtener el esfuerzo nominal a partir de la reacción.
- Leer esfuerzos en los puntos de Gauss y recuperarlos en los nodos.
- Graduar la malla hacia el concentrador de esfuerzos.
- Distinguir el error de discretización, que baja al refinar, del error de modelo, que no.

2.1 Planteamiento

Una placa de acero con un agujero de radio $a = 10$ mm se tracciona en la dirección x . Lejos del agujero el esfuerzo es uniaxial y uniforme, $\sigma_{xx} = \sigma_\infty$. La placa es delgada, con espesor de 10 mm, y trabaja en esfuerzo plano, con $E = 200$ GPa y $\nu = 0.3$.

El problema es simétrico respecto a los ejes x e y , así que basta con el cuarto $x \geq 0$, $y \geq 0$. En los bordes de simetría se impone desplazamiento normal nulo: $u_x = 0$ sobre $x = 0$ y $u_y = 0$ sobre $y = 0$. El cuarto de placa mide $W = H = 20a$, lo bastante grande para que los bordes apenas perturben el campo del agujero.

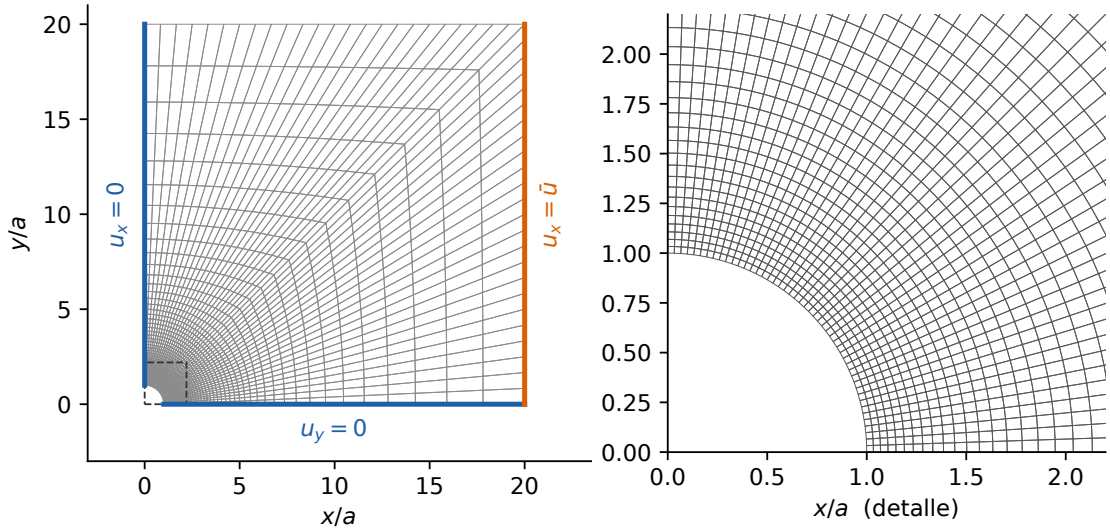


Figura 2.1: Cuarto de placa modelado: malla estructurada de cuadriláteros, condiciones de simetría (azul) y desplazamiento impuesto (naranja). A la derecha, el detalle marcado junto al agujero.

2.2 Solución analítica

Para una placa infinita con un agujero libre de tracción, en coordenadas polares (r, θ) con θ medido desde el eje x , Kirsch obtuvo

$$\begin{aligned}\sigma_{rr} &= \frac{\sigma_\infty}{2} \left(1 - \frac{a^2}{r^2} \right) + \frac{\sigma_\infty}{2} \left(1 - \frac{4a^2}{r^2} + \frac{3a^4}{r^4} \right) \cos 2\theta, \\ \sigma_{\theta\theta} &= \frac{\sigma_\infty}{2} \left(1 + \frac{a^2}{r^2} \right) - \frac{\sigma_\infty}{2} \left(1 + \frac{3a^4}{r^4} \right) \cos 2\theta, \\ \sigma_{r\theta} &= -\frac{\sigma_\infty}{2} \left(1 + \frac{2a^2}{r^2} - \frac{3a^4}{r^4} \right) \sin 2\theta.\end{aligned}$$

En el borde del agujero sólo queda el esfuerzo circunferencial, $\sigma_{\theta\theta}(a, \theta) = \sigma_\infty (1 - 2 \cos 2\theta)$. Vale $3\sigma_\infty$ en $\theta = 90^\circ$, el punto $(0, a)$, y $-\sigma_\infty$ en $\theta = 0$: el borde del agujero se comprime en la dirección transversal a la carga. Sobre el eje $x = 0$,

$$\sigma_{xx}(0, y) = \sigma_\infty \left(1 + \frac{a^2}{2y^2} + \frac{3a^4}{2y^4} \right),$$

que decae de $3\sigma_\infty$ a σ_∞ en pocos radios. El factor de concentración de esfuerzos es $K_t = \sigma_{xx}(0, a)/\sigma_\infty = 3$, independiente del tamaño del agujero y del material.

2.3 Modelo en Solidum

La malla. El script `malla.py` genera la malla con el API de Python de Gmsh y la guarda en `placa_agujero.msh`, que se versiona junto al ejemplo; resolverlo no requiere Gmsh. La malla es

estructurada y tiene dos zonas. Hasta $r = 5a$ es un anillo polar: líneas radiales rectas y circunferencias concéntricas, con elementos que crecen en progresión geométrica desde el agujero, donde el gradiente de esfuerzos es fuerte. De ahí al contorno, dos parches de transición. Todos los elementos son cuadriláteros `Quad4`. La malla define grupos físicos con nombre (`Simetria_X`, `Simetria_Y`, `Borde_Carga`, `Agujero`), y el YAML los usa para aplicar las condiciones de contorno.

La carga. El YAML de Solidum todavía no admite tracciones distribuidas sobre un borde; el API de Python sí, con el método `compute_edge_traction` de los elementos sólidos. Aquí la carga es un desplazamiento impuesto $\bar{u} = 0.1$ mm en todo el borde $x = W$, como el que aplican las mordazas de una máquina de ensayo. El esfuerzo nominal lejano se obtiene después: $\sigma_\infty = R_x/(Ht)$, con R_x la reacción total en ese borde.

```
# SOLIDUM FEM - Manual de ejemplos, ejemplo 2
# Placa con agujero circular a tracción: concentración de esfuerzos (Kirsch)
# Unidades: m, N, Pa
#
# Cuarto de placa por doble simetría: agujero de radio a = 10 mm, placa de
# 200 mm × 200 mm (20a × 20a), espesor 10 mm, acero en esfuerzo plano.
# La carga se aplica como desplazamiento impuesto en x = W; el esfuerzo
# nominal  $\sigma_\infty$  se obtiene después de la reacción total en ese borde.

mesh: placa_agujero.msh          # generada por malla.py (Gmsh), Quad4
mesh_material: 1
mesh_thickness: 0.01

materials:
  - {id: 1, type: Elastic2D, E: 200.0e9, nu: 0.3, hypothesis: plane_stress}

boundary_conditions_by_group:
  - {group_name: "Simetria_X", ux: 0.0}      # x = 0: simetría respecto al eje y
  - {group_name: "Simetria_Y", uy: 0.0}      # y = 0: simetría respecto al eje x
  - {group_name: "Borde_Carga", ux: 1.0e-4}  # x = W: alargamiento impuesto de 0.1 mm

solver:
  type: LinearSolver
```

Los esfuerzos. El método de desplazamientos da los esfuerzos con más precisión en los puntos de Gauss que en los nodos. El script los evalúa en los cuatro puntos de Gauss de cada elemento con `gauss_states` y compara cada uno con el campo de Kirsch en ese mismo punto. Para los valores en la frontera, como K_t o el esfuerzo en el borde del agujero, los extrapola a los nodos con el campo bilineal que pasa por los cuatro valores de Gauss y promedia entre los elementos que comparten el nodo. Es la recuperación nodal habitual de los programas comerciales. La función `calcular` de `run.py` hace todo el recorrido:

```
def calcular() -> dict:
    domain, _, res = resolver_yaml_estatico(AQUI / "modelo.yaml")

    # Esfuerzo nominal lejano: reacción total en el borde cargado / área.
    grupo = domain.physical_groups
    rx = sum(res.reactions_by_node[n]["ux"] for n in grupo["Borde_Carga"])
    s_inf = rx / (H * ESPESOR)

    # Puntos de Gauss: Solidum frente a Kirsch en el mismo punto.
    estados = gauss_states(domain, res.U)
```

```

pts = np.vstack([gs["points_global"] for gs in estados.values()])
sig = np.vstack([gs["stress"] for gs in estados.values()])
sk = np.column_stack(kirsch(pts[:, 0], pts[:, 1], s_inf))
err_pg = np.abs(sig - sk).max(axis=1) / s_inf
r_pg = np.hypot(pts[:, 0], pts[:, 1])
cercano = r_pg <= R_CERCANO
lejano = r_pg > R_LEJANO

# Esfuerzos nodales recuperados: línea x = 0 y borde del agujero.
nodal = esfuerzos_nodales(domain, estados)
coords = {nid: np.asarray(domain.nodes[nid].coordinates) for nid in nodal}

eje = sorted(grupo["Simetria_X"], key=lambda n: coords[n][1])
y_eje = np.array([coords[n][1] for n in eje])
sxx_eje = np.array([nodal[n][0] for n in eje])

borde = sorted(grupo["Agujero"], key=lambda n: np.arctan2(coords[n][1], coords[n][0]))
th = np.array([np.arctan2(coords[n][1], coords[n][0]) for n in borde])
s_b = np.array([nodal[n] for n in borde])
c, s = np.cos(th), np.sin(th)
stt_b = s_b[:, 0] * s**2 + s_b[:, 1] * c**2 - 2 * s_b[:, 2] * s * c
stt_k = s_inf * (1 - 2 * np.cos(2 * th))

kt = sxx_eje[0] / s_inf          # nodo (0, a): primero de la línea x = 0
return {
    "a": A, "W": W, "H": H, "espesor": ESPESOR,
    "n_elementos": len(domain.elements), "n_nodos": len(domain.nodes),
    "n_puntos_gauss": int(len(pts)), "n_puntos_cercanos": int(cercano.sum()),
    "reaccion_total": rx, "sigma_inf": s_inf, "sigma_inf_MPa": s_inf / 1e6,
    "Kt": kt, "error_Kt": error_relativo(kt, 3.0),
    "error_Kt_pct": 100 * (kt - 3.0) / 3.0,
    "sigma_min_borde": float(stt_b.min() / s_inf),
    "error_borde_max": float(np.abs(stt_b - stt_k).max() / s_inf),
    "error_borde_max_pct": float(100 * np.abs(stt_b - stt_k).max() / s_inf),
    "error_gauss_cercano": float(err_pg[cercano].max()),
    "error_gauss_cercano_pct": float(100 * err_pg[cercano].max()),
    "rms_gauss_cercano": float(np.sqrt(np.mean(err_pg[cercano] ** 2))),
    "rms_gauss_cercano_pct": float(100 * np.sqrt(np.mean(err_pg[cercano] ** 2))),
    "error_gauss_lejano": float(err_pg[lejano].max()),
    "error_gauss_lejano_pct": float(100 * err_pg[lejano].max()),
    "primer_elemento_radial_sobre_a": float(
        (np.sort(np.hypot(*np.array([domain.nodes[n].coordinates
                                     for n in grupo["Simetria_Y"]]).T))[1] - A) / A
    ),
    "eje_y": y_eje, "eje_sxx": sxx_eje,
    "borde_theta": th, "borde_stt": stt_b,
}

```

2.4 Resultados

La reacción da $\sigma_\infty = 99.41$ MPa, algo menos que los 100 MPa de $E\bar{u}/W$, porque el agujero hace a la placa ligeramente más flexible.

Tabla 2.1: Placa con agujero: Solidum frente a Kirsch. Errores relativos a σ_∞ .

Magnitud	Solidum	Kirsch
Factor de concentración K_t	3.026	3
Mínimo de $\sigma_{\theta\theta}/\sigma_\infty$ en el borde	-1.011	-1
Error máximo de $\sigma_{\theta\theta}$ en el borde	2.6 %	—
Error máximo en puntos de Gauss con $r \leq 2a$	7.5 %	—
Error medio cuadrático en puntos de Gauss con $r \leq 2a$	2.2 %	—
Error máximo en puntos de Gauss con $r > 5a$	1.0 %	—

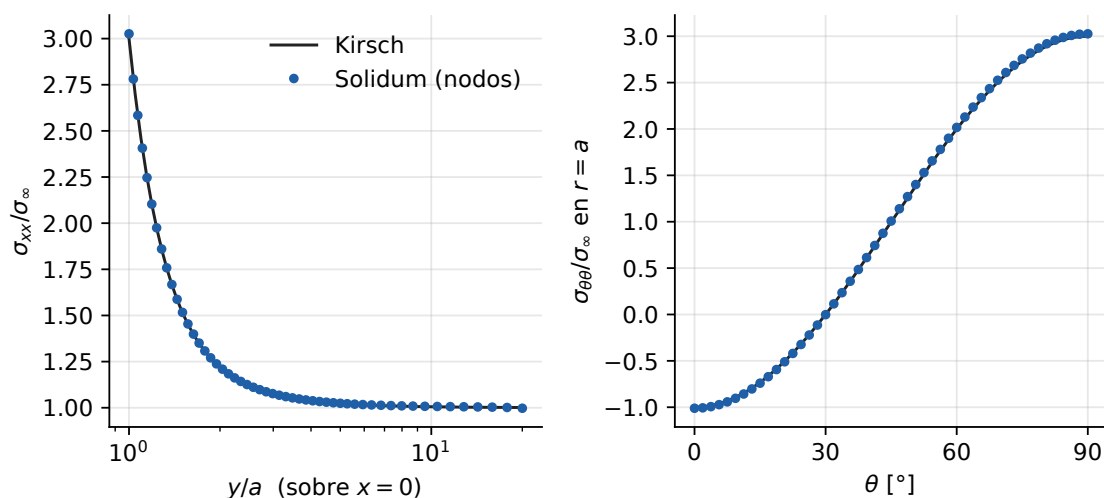


Figura 2.2: Izquierda: σ_{xx} sobre el eje $x = 0$. Derecha: esfuerzo circunferencial en el borde del agujero. Marcadores: valores nodales recuperados de Solidum.

La [figura 2.2](#) muestra las dos distribuciones que interesan en diseño. El esfuerzo sobre el eje de simetría decae de tres veces el nominal a prácticamente el nominal en unos cuatro radios, y la concentración es local. En el borde, el esfuerzo circunferencial recorre la curva $1 - 2 \cos 2\theta$ entre la compresión en $\theta = 0$ y el máximo en $\theta = 90^\circ$. El factor de concentración calculado, 3.026, está un 0.9 % por encima de 3.

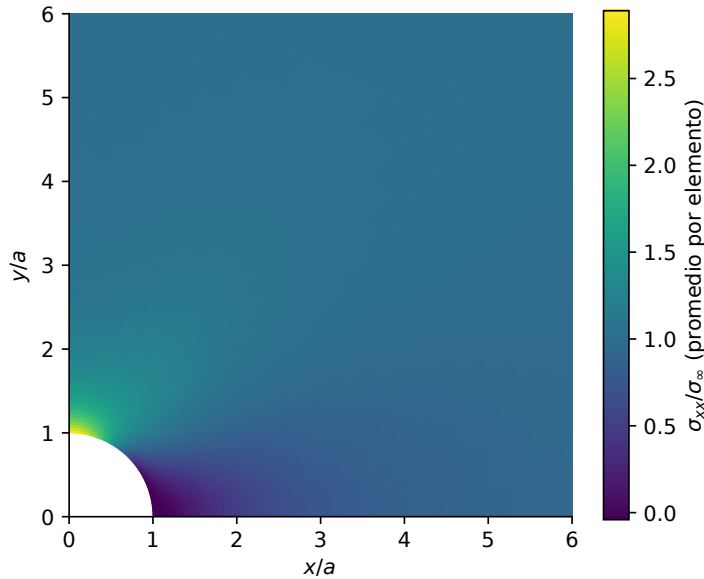


Figura 2.3: Campo de $\sigma_{xx}/\sigma_{\infty}$ cerca del agujero, promedio por elemento.

2.5 Dos fuentes de error

Los errores de la tabla no tienen un solo origen, y separarlos es lo que este ejemplo enseña.

- **Error de discretización.** Es el mayor, y se concentra junto al agujero, donde el esfuerzo varía más rápido. Un Quad4 representa los esfuerzos sólo de forma aproximadamente lineal dentro de cada elemento, y junto al agujero σ_{xx} cae a la mitad en medio radio. Por eso el primer anillo de elementos tiene un espesor radial de sólo $0.034a$. Este error baja al refinar: al duplicar la densidad de la malla, el error en los puntos de Gauss cercanos se reduce a la mitad y K_t se acerca a 3. El ejemplo 3 estudia esa convergencia en detalle.
- **Error de modelo.** Lejos del agujero el error se estanca en torno al 1 %, y refinar no lo reduce. Kirsch supone una placa infinita traccionada por un esfuerzo uniforme; el modelo es una placa finita de $20a \times 20a$ cuyo borde se desplaza uniformemente. Un desplazamiento uniforme no produce una tracción uniforme cuando el agujero perturba el campo. Las dos soluciones responden a problemas distintos, y la diferencia no es un error del programa.

La calidad de la malla también cuenta. Una primera versión de este ejemplo usaba dos parches transfinitos desde el agujero hasta el contorno, sin anillo polar. Sus líneas circunferenciales quedaban quebradas sobre la diagonal y los errores junto a ella eran bastante mayores. El anillo polar mantiene los elementos casi rectangulares donde más importa.

2.6 Para seguir

- Refinar la malla cambiando `N_ARCO`, `N_ANILLO` y `N_EXT` en `malla.py`, regenerarla y volver a resolver: K_t debe acercarse a 3 y el error junto al agujero debe bajar a la mitad con cada duplicación.

- Agrandar la placa, con W y H , para ver cómo baja el error lejano.
- Cambiar el agujero por una elipse de semiejes b en x y c en y , y comparar con la solución de Inglis, $K_t = 1 + 2c/b$.

Archivos del ejemplo. La carpeta `examples/placa_agujero_kirsch/` contiene el generador de malla `malla.py`, la malla `placa_agujero.msh`, el modelo `modelo.yaml`, el script `run.py`, los resultados en `resultados.json` y las figuras.

Capítulo 3

Cilindro de pared gruesa: estudio de convergencia con el API de Python

Un modelo de elementos finitos converge a la solución exacta cuando la malla se refina, y lo hace con una rapidez que la teoría predice para cada elemento. Este ejemplo lo comprueba en el cilindro de pared gruesa bajo presión interna, cuya solución cerrada se debe a Lamé. El modelo se construye con el API de Python, sin YAML, lo que permite generar las mallas y repetir el cálculo en un bucle.

Qué se aprende

- Construir un modelo sólido 2D con el API: nodos, elementos, material y apoyos.
- Aplicar una presión sobre un contorno curvo con su vector de cargas consistente.
- Medir errores contra una solución analítica y estimar el orden de convergencia.
- Por qué el esfuerzo converge un orden más despacio que el desplazamiento.
- Cuánto más rinden los elementos cuadráticos que los lineales por grado de libertad.

3.1 Planteamiento

Un tubo largo de acero, con radio interior $R_i = 100$ mm y exterior $R_e = 200$ mm, soporta una presión interna $p = 10$ MPa. Por ser largo y estar impedido axialmente, trabaja en deformación plana. El material tiene $E = 200$ GPa y $\nu = 0.3$. Por simetría se modela un cuarto de la sección, con $u_y = 0$ sobre el eje x y $u_x = 0$ sobre el eje y .

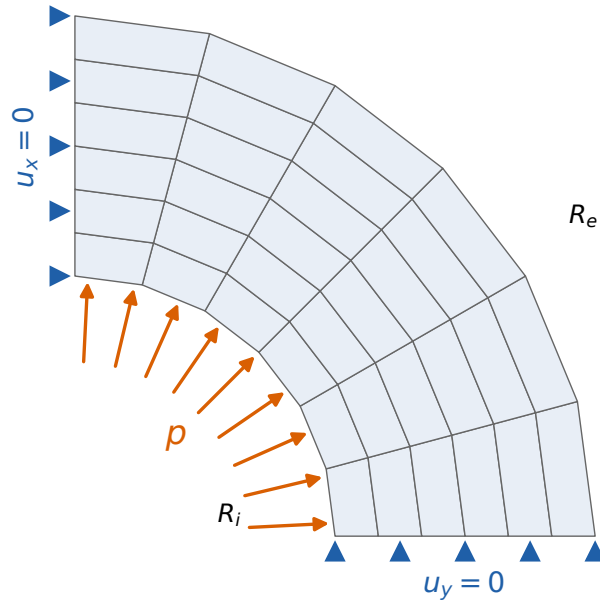


Figura 3.1: Cuarto de sección modelado, con una malla de 6×6 cuadriláteros. La presión actúa sobre el contorno interior.

3.2 Solución analítica

La solución de Lamé en coordenadas polares es

$$\sigma_{rr} = A - \frac{B}{r^2}, \quad \sigma_{\theta\theta} = A + \frac{B}{r^2}, \quad A = \frac{pR_i^2}{R_e^2 - R_i^2}, \quad B = \frac{pR_i^2 R_e^2}{R_e^2 - R_i^2},$$

y, en deformación plana,

$$u_r = \frac{1 + \nu}{E} \left[(1 - 2\nu) A r + \frac{B}{r} \right], \quad \sigma_{zz} = \nu (\sigma_{rr} + \sigma_{\theta\theta}) = 2\nu A.$$

El esfuerzo circunferencial es máximo en la cara interior, $\sigma_{\theta\theta}(R_i) = 16.67$ MPa, y el desplazamiento radial allí vale $9.533 \mu\text{m}$. El esfuerzo axial σ_{zz} es constante: Solidum lo calcula en el material, aunque el modelo 2D no lo muestra entre sus tres componentes.

3.3 Modelo con el API de Python

La función `modelo` construye el cuarto de corona con $n \times n$ celdas en (r, θ) y cualquiera de los cuatro elementos del estudio: `Tri3` y `Quad4`, lineales, y `Tri6` y `Quad8`, cuadráticos. Los nodos intermedios de los elementos cuadráticos quedan sobre las circunferencias, así que su contorno interior es curvo; el de los lineales es un polígono inscrito.

```

def modelo(nombre: str, n: int, material=None, cuadratura: str | None = None):
    """Cuarto de corona con  $n \times n$  celdas de ``nombre``. Devuelve el dominio,
    las aristas cargadas ``[(elemento, arista)]`` y los nodos. Por omisión,
    acero elástico en deformación plana y la cuadratura propia de cada
    elemento (el ejemplo 4 pasa un material plástico y, para el Quad8,
    integración reducida ``"2x2"``)."""
    cls = CLASES[nombre]
    cuadratico = nombre in ("Tri6", "Quad8")
    k = 2 if cuadratico else 1          # nodos por celda en cada dirección
    ni = nj = k * n + 1
    if material is None:
        material = Elastic2D(E=E, nu=NU, hypothesis="plane_strain")
    domain = Domain()
    nodo = {}
    for j in range(nj):
        th = 0.5 * np.pi * j / (nj - 1)
        for i in range(ni):
            if nombre == "Quad8" and i % 2 == 1 and j % 2 == 1:
                continue                # Quad8 no tiene nodo central
            r = RI + (RE - RI) * i / (ni - 1)
            nodo[i, j] = domain.add_node(len(nodo) + 1, [r * np.cos(th), r * np.sin(th)
                ])

    aristas = []
    for cj in range(n):
        for ci in range(n):
            i, j = k * ci, k * cj
            c1, c2, c3, c4 = nodo[i, j], nodo[i + k, j], nodo[i + k, j + k], nodo[i, j
                + k]
            eid = len(domain.elements) + 1
            if nombre == "Quad4":
                el = [cls(eid, [c1, c2, c3, c4], material, thickness=1.0,
                    quadrature=cuadratura)]
                cara = (el[0], 3)                # arista c4 → c1, en r = RI
            elif nombre == "Quad8":
                m = [nodo[i + 1, j], nodo[i + 2, j + 1], nodo[i + 1, j + 2], nodo[i, j
                    + 1]]
                el = [cls(eid, [c1, c2, c3, c4] + m, material, thickness=1.0,
                    quadrature=cuadratura)]
                cara = (el[0], 3)
            elif nombre == "Tri3":
                el = [cls(eid, [c1, c2, c3], material, thickness=1.0),
                    cls(eid + 1, [c1, c3, c4], material, thickness=1.0)]
                cara = (el[1], 2)                # arista c4 → c1
            else:
                cen = nodo[i + 1, j + 1]
                el = [cls(eid, [c1, c2, c3, nodo[i + 1, j], nodo[i + 2, j + 1], cen],
                    material, thickness=1.0),
                    cls(eid + 1, [c1, c3, c4, cen, nodo[i + 1, j + 2], nodo[i, j +
                        1]],
                    material, thickness=1.0)]
                cara = (el[1], 2)
            for e in el:
                domain.add_element(e)
            if ci == 0:

```

```

        aristas.append(cara)

    for i in range(ni):
        # simetría
        if (i, 0) in nodo:
            nodo[i, 0].fix_dof("uy", 0.0)
        if (i, nj - 1) in nodo:
            nodo[i, nj - 1].fix_dof("ux", 0.0)
    domain.generate_equation_numbers()
    return domain, aristas, nodo

```

La presión. El método `compute_edge_traction` de los elementos sólidos integra una tracción de dirección constante sobre una arista. Una presión no lo es: empuja en la dirección normal al contorno, que cambia a lo largo de una arista curva. La función `carga_de_presion` integra $\int_{\Gamma} \mathbf{N}^T (-p \mathbf{n}) d\Gamma$ con tres puntos de Gauss sobre la geometría de cada arista, recta o parabólica:

```

def carga_de_presion(domain, aristas, p: float) -> np.ndarray:
    """Vector de cargas consistente con una presión ``p`` sobre las aristas.

    La presión empuja contra la cara:  $\mathbf{t} = -p \mathbf{n}$ , con  $\mathbf{n}$  la normal exterior del
    sólido. Se integra  $\int \mathbf{N}^T \mathbf{t} d\Gamma$  sobre la geometría discretizada de cada
    arista (recta o parabólica), con tres puntos de Gauss:  $\mathbf{n}$  varía a lo largo
    de una arista curva, así que no sirve una tracción constante.
    """
    F = np.zeros(domain.total_dofs)
    xg, wg = np.polynomial.legendre.leggauss(3)
    for el, arista in aristas:
        loc = el.EDGE_NODES[arista]
        X = np.array([el.nodes[q].coordinates[:2] for q in loc], dtype=float)
        dofs = el.get_global_dof_indices()
        for s, w in zip(xg, wg):
            if len(loc) == 2:
                N = np.array([(1 - s) / 2, (1 + s) / 2])
                dN = np.array([-0.5, 0.5])
            else:
                # (extremo, medio, extremo)
                N = np.array([s * (s - 1) / 2, 1 - s**2, s * (s + 1) / 2])
                dN = np.array([s - 0.5, -2 * s, s + 0.5])
            dx, dy = dN @ X
            # Recorriendo la arista en el sentido del elemento (antihorario), la
            # normal exterior multiplicada por |J| es (dy, -dx).
            t = -p * np.array([dy, -dx])
            for a, q in enumerate(loc):
                for d in range(2):
                    g = dofs[2 * q + d]
                    if g >= 0:
                        F[g] += w * N[a] * t[d] * el.thickness
    return F

```

Con el dominio y la carga, resolver es una línea: `LinearSolver(Assembler(domain)).solve(F)` devuelve el vector de desplazamientos.

3.4 Estudio de convergencia

Cada elemento se resuelve en mallas de 2×2 a 64×64 celdas para los lineales, y hasta 32×32 para los cuadráticos, que llegan antes a errores muy pequeños. Se miden dos errores relativos:

- en desplazamientos, la media cuadrática del error de u_r en todos los nodos;
- en esfuerzos, la media cuadrática del error de $(\sigma_{rr}, \sigma_{\theta\theta})$ en todos los puntos de Gauss.

Si el error se comporta como $e \approx C h^k$, en escala logarítmica es una recta de pendiente k , el **orden de convergencia**. La teoría del método predice, para una interpolación de grado p , orden $p+1$ en desplazamientos y p en esfuerzos: los esfuerzos salen de derivar los desplazamientos y pierden un orden.

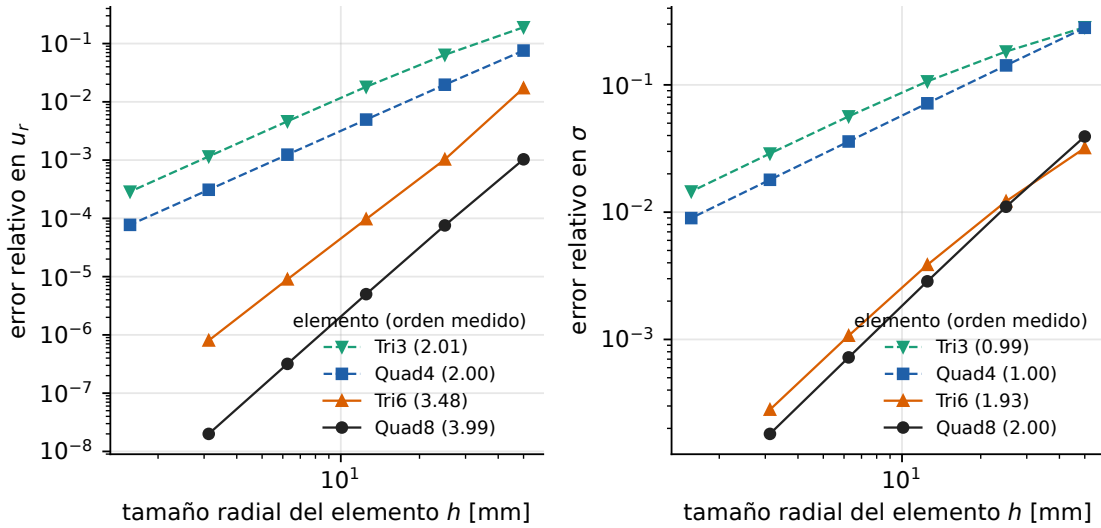


Figura 3.2: Error relativo frente al tamaño radial del elemento. Entre paréntesis, la pendiente medida entre las dos mallas más finas.

Tabla 3.1: Órdenes de convergencia medidos entre las dos mallas más finas y órdenes teóricos.

Elemento	p	Desplazamiento, medido	Desplazamiento, teórico	Esfuerzo, medido	Esfuerzo, teórico
Tri3	1	2.01	2	0.99	1
Quad4	1	2.00	2	1.00	1
Tri6	2	3.48	3	1.93	2
Quad8	2	3.99	3	2.00	2

Los esfuerzos convergen con el orden teórico en los cuatro elementos. Los desplazamientos lo alcanzan o lo superan. El Quad8 llega a orden 4: en mallas regulares, los valores nodales de algunos elementos convergen más rápido que el campo en su conjunto, un fenómeno conocido como **superconvergencia**. La teoría garantiza el orden $p+1$; lo que se mide por encima es un regalo de la regularidad de la malla, con el que no conviene contar en mallas arbitrarias.

3.5 Lineales frente a cuadráticos

La figura 3.2 muestra la diferencia práctica. Con la malla de 8×8 celdas, el error en esfuerzos del Quad4 es 7.2 % y el del Quad8 0.29 %. Con la malla más fina, de 8450 grados de libertad, el Quad4 todavía queda en 0.90 %, peor que el Quad8 de 8×8 con sólo 450 grados de libertad. Cuando el campo de esfuerzos es suave, los elementos cuadráticos alcanzan una precisión dada con muchos menos grados de libertad.

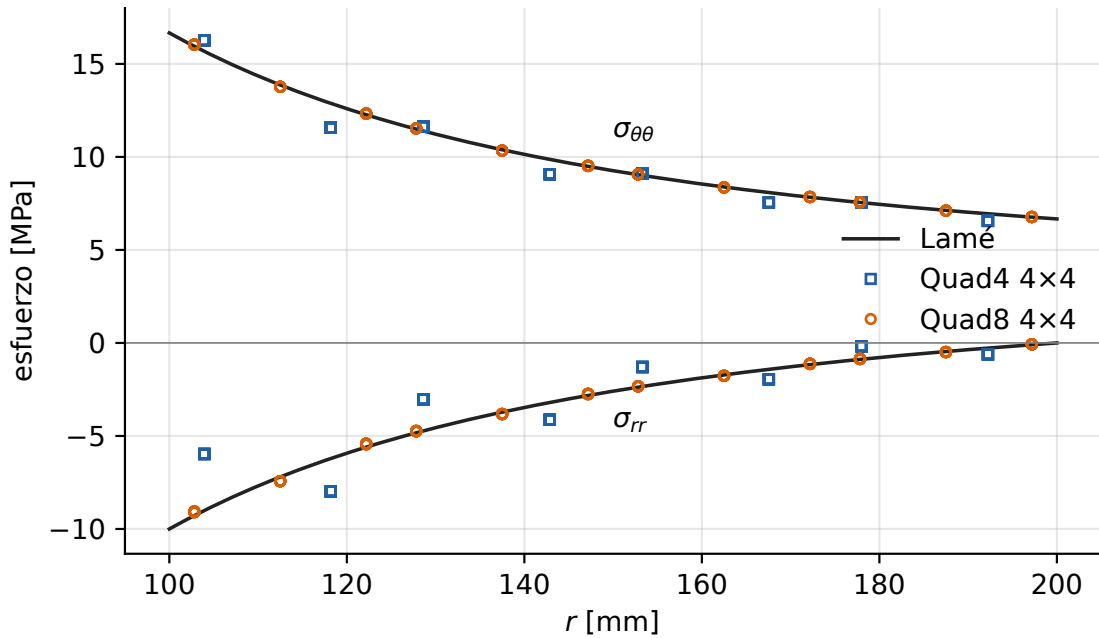


Figura 3.3: Esfuerzos en los puntos de Gauss con una malla de 4×4 celdas.

La figura 3.3 explica el motivo. Con cuatro Quad4 en el espesor, el esfuerzo de cada elemento no puede seguir la variación $1/r^2$: sus puntos se dispersan alrededor de la curva, sobre todo en σ_{rr} cerca de la cara interior. El Quad8, cuyo esfuerzo varía linealmente dentro del elemento, reproduce la solución con la misma malla.

3.6 Para seguir

- Repetir el estudio con $\nu = 0.4999$. En deformación plana, casi incompresible, los elementos lineales con integración completa sufren **bloqueo volumétrico**: el error en desplazamientos crece mucho y la solución sale demasiado rígida.
- Adaptar la función modelo al Quad9, que añade al Quad8 el nodo central de cada celda, y comparar ambos.
- Distorsionar la malla desplazando al azar los nodos interiores. ¿Se mantienen los órdenes medidos? ¿Y la superconvergencia del Quad8?

Archivos del ejemplo. La carpeta `examples/cilindro_lame/` contiene el script `run.py`, que genera las mallas, los resultados en `resultados.json` y las figuras.

Capítulo 4

Cilindro elastoplástico: primera fluencia, colapso y bloqueo volumétrico

El cilindro del ejemplo anterior, ahora de un material que fluye. Al aumentar la presión, la cara interior plastifica primero; la zona plástica avanza hacia fuera y, cuando alcanza la cara exterior, el tubo ya no admite más presión: ha llegado al colapso plástico. Las dos presiones que marcan ese recorrido tienen expresión cerrada, y el ejemplo las usa para verificar el material de von Mises, el solver de longitud de arco y el comportamiento de cinco elementos en régimen plástico.

Qué se aprende

- Usar un material elastoplástico (**VonMises2D**) en deformación plana.
- Trazar una curva presión-desplazamiento con el método de longitud de arco, que puede seguir la meseta del colapso.
- Declarar el primer paso como fracción de la carga, sin conocer de antemano los desplazamientos.
- Reconocer el **bloqueo volumétrico** en plasticidad y evitarlo con la elección del elemento.

4.1 Planteamiento

La geometría, la malla y la carga son las del ejemplo 3: radio interior $a = 100$ mm, exterior $b = 200$ mm, un cuarto de sección por simetría y presión interna p . El acero es ahora elastoplástico perfecto, sin endurecimiento: $E = 200$ GPa, $\nu = 0.3$ y esfuerzo de fluencia $\sigma_y = 240$ MPa, con el criterio de von Mises. El tubo trabaja en deformación plana.

4.2 Soluciones analíticas

Primera fluencia. Mientras el material es elástico vale la solución de Lamé. El esfuerzo equivalente de von Mises es máximo en la cara interior, donde, con $\sigma_{zz} = \nu(\sigma_{rr} + \sigma_{\theta\theta})$,

$$\sigma_{eq}(a) = \frac{p}{b^2 - a^2} \sqrt{3b^4 + (1 - 2\nu)^2 a^4}, \quad p_e = \sigma_y \frac{b^2 - a^2}{\sqrt{3b^4 + (1 - 2\nu)^2 a^4}}.$$

Con los datos del ejemplo, $p_e = 103.75$ MPa.

Colapso. En el colapso toda la pared fluye y el flujo plástico no cambia el volumen. En deformación plana eso impone $\sigma_{zz} = (\sigma_{rr} + \sigma_{\theta\theta})/2$, y el criterio de von Mises se reduce a $\sigma_{\theta\theta} - \sigma_{rr} = 2\sigma_y/\sqrt{3}$. Integrando el equilibrio radial, $d\sigma_{rr}/dr = (\sigma_{\theta\theta} - \sigma_{rr})/r$, entre la cara interior, con $\sigma_{rr} = -p$, y la exterior, libre, resulta

$$p_{\text{lim}} = \frac{2}{\sqrt{3}} \sigma_y \ln \frac{b}{a}.$$

Vale 192.09 MPa. Es una carga límite en sentido estricto: por encima de ella no existe equilibrio, y por los teoremas del análisis límite no depende de E ni de ν . La primera fluencia ocurre a una fracción 0.540 de ella.

4.3 Modelo en Solidum

El modelo reutiliza las funciones `modelo` y `carga_de_presion` del ejemplo 3, con el material plástico:

```
def trazar(elemento: str, n: int, cuadratura, *, dlambda: float = DLAMBDA,
           dl: float | None = None, pasos: int = PASOS):
    """Curva  $\lambda(, u_b)$  con  $p = \lambda \cdot p_{\text{lim}}$ ,  $u_b$  = desplazamiento radial exterior.

    El primer paso se declara como fracción de la carga (dlambda) o, si
    se pasa dl, como longitud de arco explícita en metros. Devuelve
    también el esfuerzo medio de los puntos de Gauss en el primer paso con
     $u_b \geq 1$  mm (None si no se llega), un punto fijo del recorrido
    comparable entre elementos, y la longitud de arco del primer paso."""
    material = VonMises2D(E=E, nu=NU, sigma_y=SY, H=0.0, hypothesis="plane_strain")
    domain, aristas, nodo = modelo(elemento, n, material=material, cuadratura=
        cuadratura)
    F = carga_de_presion(domain, aristas, P_LIM)          #  $\lambda = 1 \Leftrightarrow$  presión de colapso
    assembler = Assembler(domain)
    primer_paso = {"initial_dl": dl} if dl is not None else {"initial_dlambda": dlambda
    }
    solver = ArcLengthSolver(assembler, max_iter=30, max_lambda=1.5,
                             max_steps=pasos, **primer_paso)
    exterior = nodo[max(i for i, _ in nodo), 0]          # (b, 0)
    curva, medio = [], []

    def al_converger(paso, U, lam):
        # El solver llama aquí con el estado ya consolidado (commit).
        curva.append((lam, U[exterior.dofs["ux"]]))
        if not medio and curva[-1][1] >= U_LECTURA[1]:
            medio.append(presion_medio(domain))

    solidum.run(domain, assembler=assembler, solver=solver, F_applied=F,
                step_callback=al_converger)
    lam, ub = np.array(curva).T
    return domain, lam, ub, (medio[0] if medio else None), solver.dl_reference
```

La carga de referencia es la presión de colapso, de modo que el factor de carga λ que devuelve el solver es directamente p/p_{lim} .

Por qué longitud de arco. El solver de Newton con control de carga ([NonlinearSolver](#)) impone el valor de λ en cada paso. Cerca del colapso la curva se vuelve horizontal y más allá no hay equilibrio, así que ese esquema no puede recorrerla. El [ArcLengthSolver](#) convierte λ en una incógnita más y avanza una distancia fija, la longitud de arco, en el espacio de desplazamientos y carga. Así sigue la meseta con la carga prácticamente constante y el desplazamiento creciente. Como la meseta nunca llega a `max_lambda`, el recorrido termina al agotar `max_steps`, y el solver lo avisa. Si un paso cruzara `max_lambda`, el solver lo repetiría hasta llegar a ese valor exacto dentro del tramo recién recorrido: nunca impone la carga máxima sin haber seguido la curva hasta ella.

El primer paso. La longitud de arco es una longitud en unidades de desplazamiento, y quien plantea el modelo no conoce de antemano cuánto se desplazará la estructura. Por eso el primer paso se declara con `initial_dlambda`, como fracción de la carga de referencia: aquí 0.02, el 2 % de la presión de colapso. El solver lo convierte en longitud de arco con una resolución elástica, $\Delta l_1 = \Delta \lambda_1 \|\mathbf{K}^{-1} \mathbf{F}_{\text{ref}}\|$, que aquí da 2.3×10^{-5} m. El valor por omisión es 0.1, y vale igual en cualquier sistema de unidades.

Advertencia

El parámetro alternativo, `initial_dl`, fija la longitud de arco del primer paso directamente, en unidades de desplazamiento, y para elegirlo hay que conocer la escala del problema. Aquí, 0.1 m equivale a un primer paso de 86 veces la presión de colapso. Con ese valor el primer paso llega a un desplazamiento exterior de 8.6 mm, 74 veces el elástico en el colapso: se salta toda la transición elastoplástica. Cinco pasos después el tubo se ha desplazado 113 mm, con una carga 1.058 veces la de colapso. Ese exceso sobre p_{lim} no es un error del solver: es el Quad8 con integración completa, que a desplazamientos tan grandes también se bloquea (ver más abajo), y el solver sigue fielmente la curva del modelo discreto. En las mismas condiciones, el Quad8 con integración 2×2 se queda en 1.0000 veces la presión de colapso.

4.4 Resultados

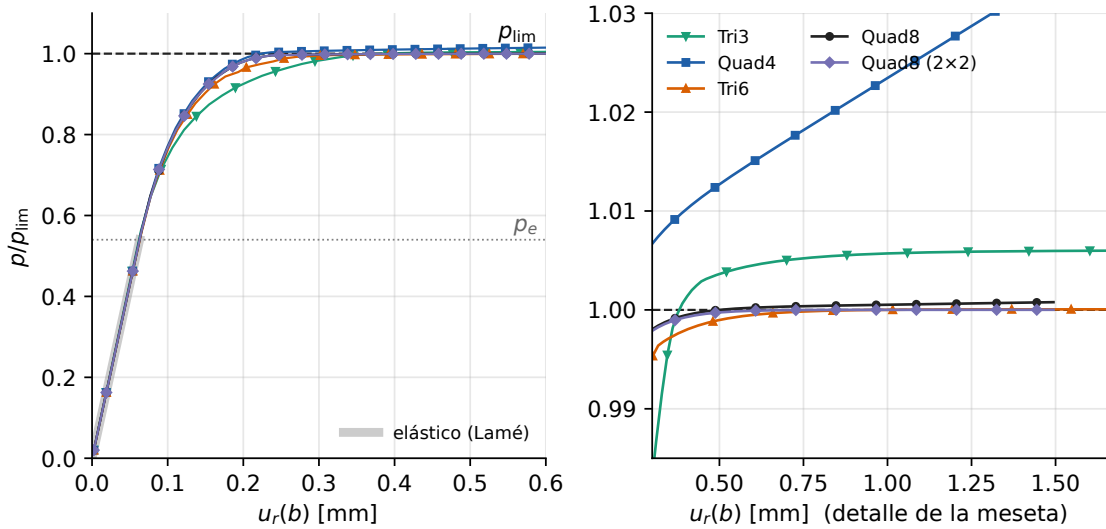


Figura 4.1: Presión frente al desplazamiento radial exterior. Izquierda: curva completa. Derecha: detalle de la meseta de colapso, con la escala vertical ampliada.

Tabla 4.1: Cilindro elastoplástico. Error de la flexibilidad elástica frente a Lamé y carga relativa p/p_{lim} con desplazamientos exteriores de 0.5 y 1 mm.

Elemento	Grados de libertad	Error elástico	p/p_{lim} a 0.5 mm	p/p_{lim} a 1 mm
Tri3, 8×8	162	1.8×10^{-2}	1.0036	1.0057
Quad4, 8×8	162	4.3×10^{-3}	1.0127	1.0234
Tri6, 4×4	162	3.0×10^{-5}	0.9990	1.0000
Quad8, 4×4	130	5.3×10^{-5}	1.0000	1.0005
Quad8 con 2×2 , 4×4	130	5.0×10^{-5}	0.9997	1.0000

La figura 4.1 y la tabla muestran tres tramos:

- **Elástico.** Todas las curvas arrancan sobre la recta de Lamé. Los elementos cuadráticos reproducen su pendiente con un error del orden de 10^{-5} ; el Tri3, el más rígido, la sobrestima un 1.8×10^{-2} .
- **Elastoplástico.** Las curvas se separan de la recta a partir de p_e , cuando la cara interior empieza a fluir, y se curvan a medida que la zona plástica avanza.
- **Colapso.** Tri6 y los dos Quad8 se aplanan sobre p_{lim} : a 1 mm de desplazamiento la presión está a menos de 10^{-3} de la analítica. El Tri3 también forma una meseta, un 5.7×10^{-3} por encima. El Quad4 no la forma: gana un 1.1×10^{-2} de carga entre 0.5 y 1 mm, y sigue subiendo hasta el final del recorrido.

4.5 Bloqueo volumétrico

La deformación plástica no cambia el volumen. En el colapso, el mecanismo del tubo es un flujo radial isocórico, y el modelo sólo puede seguirlo si sus funciones de forma admiten un campo de desplazamiento sin cambio de volumen en los puntos de integración que fluyen. El Quad4 con integración completa impone esa condición en sus cuatro puntos de Gauss, más de las que su interpolación bilineal puede satisfacer, y el mecanismo discreto resulta más rígido que el real. Es el **bloqueo volumétrico**, descrito por Nagtegaal, Parks y Rice en 1974 precisamente para el régimen totalmente plástico.

El síntoma se ve en el esfuerzo medio, $(\sigma_{xx} + \sigma_{yy} + \sigma_{zz})/3$. En el colapso exacto vale $\sigma_{rr} + \sigma_y/\sqrt{3}$, entre -54 MPa en la cara interior y 139 MPa en la exterior. Con 1 mm de desplazamiento exterior, el Quad4 tiene puntos de Gauss entre -223 y 171 MPa: una presión espuria, que el criterio de von Mises no limita porque sólo acota la parte desviadora del esfuerzo. El Tri6 y los dos Quad8 quedan dentro del intervalo exacto, salvo el error de discretización.

El bloqueo no es un error del programa ni del material: el mapeo de retorno respeta la superficie de fluencia en todos los puntos y el equilibrio se cumple. Es una propiedad del elemento, y se evita eligiéndolo bien:

- Los elementos cuadráticos, Tri6 y Quad8, no muestran bloqueo apreciable en este problema.
- El Quad8 con integración reducida 2×2 , que impone la condición en cuatro puntos en vez de nueve, da la meseta más plana. En el API se elige con el argumento `quadrature="2x2"` del elemento, y en un YAML con elementos declarados uno a uno, con la clave `quadrature` de cada elemento.
- Solidum no tiene todavía formulaciones contra el bloqueo para el Quad4, como la B-barra o las mixtas desplazamiento-presión. Para plasticidad en deformación plana, o para materiales casi incompresibles, conviene usar elementos cuadráticos.

4.6 Para seguir

- Añadir endurecimiento, $H > 0$ en `VonMises2D`, y observar que la meseta se inclina: p_{lim} deja de ser una carga límite.
- Intentar superar p_{lim} con `NonlinearSolver` y control de carga. ¿Qué hace el solver cuando no hay equilibrio?
- Refinar la malla del Quad4. ¿Desaparece el bloqueo?

Referencias. R. Hill, *The Mathematical Theory of Plasticity*, Oxford, 1950, cap. V. J. C. Nagtegaal, D. M. Parks y J. R. Rice, "On numerically accurate finite element solutions in the fully plastic range", *Computer Methods in Applied Mechanics and Engineering* 4 (1974) 153-177. E. A. de Souza Neto, D. Perić y D. R. J. Owen, *Computational Methods for Plasticity*, Wiley, 2008, §7.5.

Archivos del ejemplo. La carpeta `examples/cilindro_elastoplastico/` contiene el script `run.py`, que construye el modelo con las funciones del ejemplo 3, los resultados en `resultados.json` y la figura.

Capítulo 5

Barra con daño: localización, retroceso y control indirecto

Una barra a tracción de un material que se ablanda: pasado el máximo, cuanto más se estira, menos fuerza soporta. Es el comportamiento del hormigón, de la roca o de la madera cuando se fisuran. El ejemplo muestra por qué ese tramo descendente no se puede recorrer controlando la carga, cómo el daño se concentra en una sola zona mientras el resto se descarga, y por qué una barra larga puede **retroceder**: desplazarse hacia atrás mientras la carga cae. Todo se compara con la solución exacta.

Qué se aprende

- Usar un material de daño con ablandamiento (`IsotropicDamage1D`).
- Por qué el control de carga se detiene en el pico, y el de desplazamiento en un retroceso.
- Seguir el retroceso con `IndirectDisplacementSolver`, el **control indirecto de desplazamiento**.
- Que con ablandamiento un paso demasiado grande puede llevar a otro equilibrio.

5.1 Planteamiento

Una barra de sección $A = 100 \text{ cm}^2$ empotrada en un extremo y cargada con una fuerza F en el otro, dividida en elementos `Truss2D` de longitud $h = 0.1 \text{ m}$. El material tiene $E = 30 \text{ GPa}$ y un daño isótropo que empieza cuando la deformación alcanza $\kappa_0 = 10^{-4}$, es decir, con un esfuerzo $\sigma_t = E\kappa_0 = 3.0 \text{ MPa}$. El elemento central es un 5 % menos resistente: es la imperfección que decide dónde se localiza el daño, como en un ensayo real decide la zona más débil de la probeta.

Se estudian dos barras con el mismo tamaño de elemento: una **corta**, de 0.2 m (2 elementos), y una **larga**, de 1 m (10 elementos).

5.2 Solución analítica

El material. Con la variable de historia κ , la deformación máxima alcanzada, el daño y el esfuerzo son

$$d(\kappa) = 1 - \frac{\kappa_0}{\kappa} e^{-\alpha(\kappa - \kappa_0)} \quad (\kappa > \kappa_0), \quad \sigma = (1 - d) E \varepsilon,$$

con $\alpha = 5000$. En carga, $\sigma = E \kappa_0 e^{-\alpha(\varepsilon - \kappa_0)}$: el esfuerzo cae exponencialmente desde σ_t . En descarga, el material vuelve al origen por la secante, sin más daño.

Localización. Sin fuerzas de volumen, el equilibrio impone el mismo esfuerzo en toda la barra. La carga máxima la fija el elemento débil, $F_{\text{pico}} = A E \kappa_{0w} = 28.5$ kN, con κ_{0w} su umbral. Pasado el pico el esfuerzo baja; el elemento débil sigue dañándose, pero los demás, que nunca llegaron a su umbral, se descargan elásticamente. Con la deformación del elemento débil ε_w como parámetro, la respuesta exacta es

$$F = A \sigma(\varepsilon_w), \quad u = h \varepsilon_w + (L - h) \frac{\sigma(\varepsilon_w)}{E}.$$

Retroceso. Justo después del pico, $d\sigma/d\varepsilon_w = -\alpha \sigma$, y el desplazamiento del extremo cambia a razón de

$$\frac{du}{d\varepsilon_w} = h - (L - h) \alpha \kappa_{0w}.$$

Si es negativa, el desplazamiento **disminuye** mientras la carga cae: la curva retrocede (*snap-back*). Ocurre cuando $(L/h - 1) \alpha \kappa_{0w} > 1$. Ese producto vale 0.475 en la barra corta y 4.275 en la larga: la corta baja sin más; la larga retrocede hasta $u = 0.616 u_{\text{pico}}$ antes de volver a avanzar.

El balance de energía lo explica. El elemento débil disipa en toda su rotura 0.705 J, lo mismo en las dos barras porque tiene el mismo tamaño. En el pico, la barra corta tiene almacenados 0.271 J de energía elástica y la larga 1.354 J. Al romperse, toda esa energía elástica se libera, y la parte que el elemento no puede disipar tiene que devolverse a la carga exterior. Para eso el trabajo de la fuerza tiene que ser negativo, y con la fuerza a tracción eso sólo ocurre si el extremo retrocede. La barra corta no tiene exceso que devolver.

5.3 Modelo en Solidum

La barra se construye elemento a elemento, con el central debilitado:

```
def barra(L: float):
    """Barra de ``L/H`` elementos ``Truss2D`` con el central debilitado,
    empotrada en ``x = 0``. Devuelve el dominio, el nodo del extremo libre y
    el elemento débil; cada control añade su carga o su apoyo."""
    n = round(L / H)
    dom = Domain()
    nodos = [dom.add_node(i + 1, [i * H, 0.0]) for i in range(n + 1)]
    for k in range(n):
        umbral = KOW if k == n // 2 else KAPPA_0
        material = IsotropicDamage1D(E=E, kappa_0=umbral, alpha=ALPHA)
        dom.add_element(Truss2D(k + 1, [nodos[k], nodos[k + 1]], material, A=A))
    nodos[0].fix_dof("ux", 0.0)
    for nodo in nodos:
        nodo.fix_dof("uy", 0.0)
    extremo, debil = nodos[-1], dom.elements[n // 2 + 1]
    return dom, extremo, debil
```

Se resuelve con tres controles distintos:

- **Control de carga** ([NonlinearSolver](#) con una fuerza creciente hasta 1.2 veces la de pico). Impone la carga de cada paso, y por encima de F_{pico} no hay equilibrio.
- **Control de desplazamiento** (el mismo solver con el desplazamiento del extremo impuesto). La carga es ahora la reacción, y puede bajar; pero el desplazamiento impuesto sólo crece.
- **Control indirecto** ([IndirectDisplacementSolver](#)). Hace crecer en cada paso una magnitud elegida, aquí el alargamiento del elemento débil, y deja libres a la vez la carga y el desplazamiento del extremo. Es como se controla en laboratorio un ensayo de fractura, con la apertura de la fisura:

```
def control_indirecto(L: float, dlambda: float = DLAMBDA, pasos: int = 500):
    """Control indirecto sobre el alargamiento del elemento débil."""
    dom, extremo, debil = barra(L)
    dom.generate_equation_numbers(verbose=False)
    F = np.zeros(dom.total_dofs)
    F[extremo.dofs["ux"]] = F_PICO
    a, b = debil.nodes
    solver = IndirectDisplacementSolver(
        Assembler(dom), control=[(b, "ux", 1.0), (a, "ux", -1.0)],
        initial_dlambda=dlambda, max_lambda=1.2, max_steps=pasos, max_iter=30)
    curva: list = []
    solver.solve(F, step_callback=_registro(dom, extremo, debil, curva,
        lambda U, lam: lam * F_PICO))
    return np.array(curva)
```

La magnitud controlada se declara como una combinación de grados de libertad, $\mathbf{c}^\top \mathbf{U}$: aquí, $u_b - u_a$ entre los dos nodos del elemento débil. El primer paso es un 2% de la carga de pico, y el solver mantiene constante ese incremento de alargamiento en todo el trazado.

5.4 Resultados

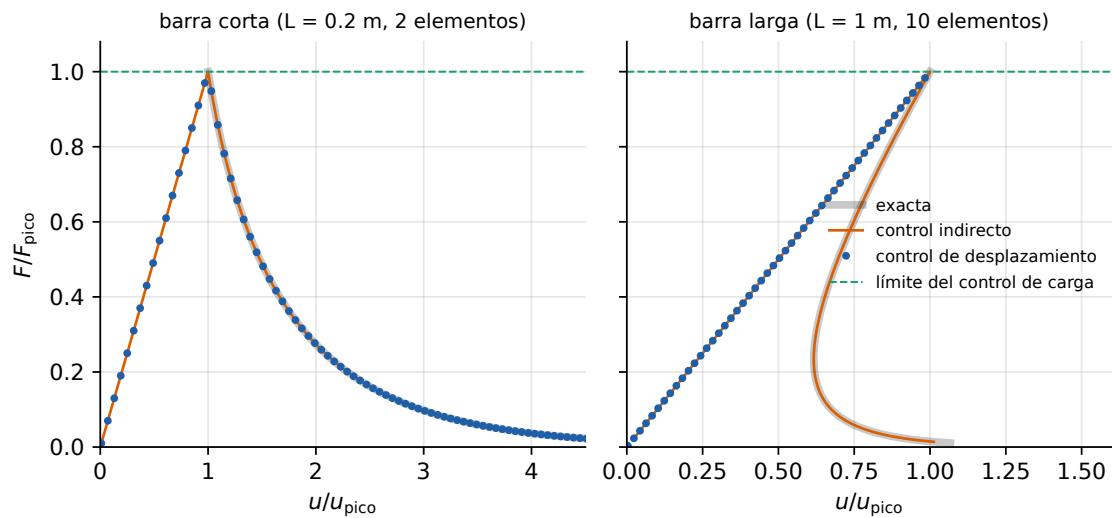


Figura 5.1: Carga frente al desplazamiento del extremo, normalizados con el pico. La curva exacta en gris.

Tabla 5.1: Qué consigue cada control en cada barra. El error se mide frente a la solución exacta con la misma deformación del elemento débil.

Control	Barra corta	Barra larga
Carga	se detiene en el pico	se detiene en el pico
Desplazamiento del extremo	curva completa, error menor que 10^{-12}	se detiene en el pico, $u = 1.000 u_{pico}$
Indirecto	curva completa, error menor que 10^{-12}	curva completa, error menor que 10^{-12} ; retrocede hasta $0.616 u_{pico}$

La [figura 5.1](#) resume los tres comportamientos:

- **El control de carga** llega hasta F_{pico} en las dos barras y ahí termina con un error de divergencia (`UnknownDivergenceError`): el solver reduce el paso sin encontrar equilibrio por encima.
- **El control de desplazamiento** recorre toda la rama descendente de la barra corta, sobre la curva exacta. En la larga se detiene en el pico: a partir de ahí el camino de equilibrio sigue con desplazamientos *menores*. Con un desplazamiento impuesto mayor que el del pico, el único equilibrio que queda en el camino físico está al otro lado del lazo, con la carga casi nula, y el solver no llega a él.
- **El control indirecto** sigue las dos curvas exactas, incluido el retroceso de la larga, con un solo elemento dañado en todo el trazado. En las dos barras llega a una carga de 0.014 veces la de pico en 500 pasos.

El arco cilíndrico de [ArcLengthSolver](#), con el mismo primer paso, tampoco sigue el retroceso: en la barra larga termina en el pico con un error (`RuntimeError`) sin haber retrocedido. Tras el pico, la parte de la barra que se descarga invierte casi por completo el sentido de su desplazamiento, y el criterio con que ese solver elige la solución, la más alineada con el paso anterior, la descarta.

Advertencia

Con ablandamiento, el problema de cada paso puede tener más de una solución. Además de la física, en la que sólo se daña el elemento débil, hay otras en las que también se dañan elementos que en realidad se habrían descargado. Cuál encuentra el solver depende de su punto de partida. Si un paso sobrepasa el pico más que la imperfección que localiza el daño, aquí un 5 %, el solver puede converger a otra. Con pasos del 10 % de la carga, el control indirecto de la barra larga acaba con los 10 elementos dañados y un desplazamiento que se aparta hasta 9 veces u_{pico} de la solución exacta (figura 5.2). No es propio de este solver: el control de desplazamiento de la barra corta con 12 pasos termina con sus 2 elementos dañados. La defensa es un paso pequeño cerca del pico; por eso `IndirectDisplacementSolver` no agranda el paso por omisión.

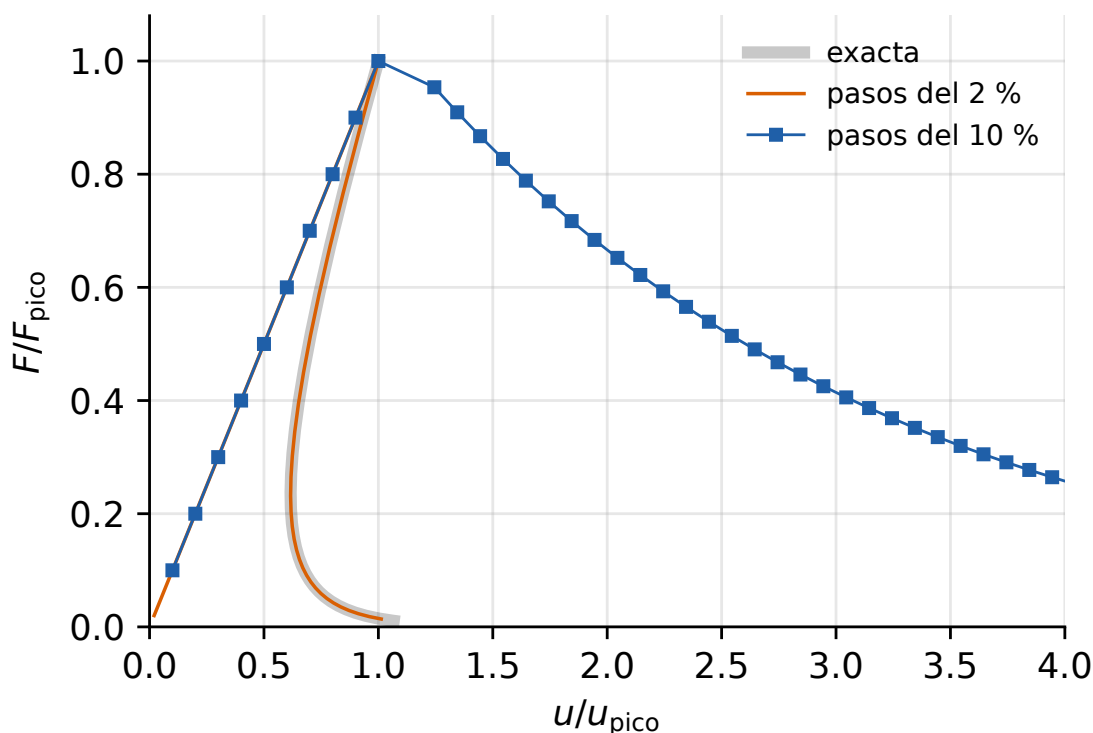


Figura 5.2: Barra larga con control indirecto: con pasos del 2 % se sigue la curva exacta; con pasos del 10 % el solver salta a otro equilibrio, con todos los elementos dañados.

5.5 Para seguir

- Refinar la malla de la barra larga, con h más pequeño. ¿Qué pasa con el retroceso? Con daño local la zona que se rompe mide siempre un elemento, y la energía disipada es proporcional a h : el resultado depende de la malla. Los modelos regularizados, o los de fisura cohesiva, que disipan una energía de fractura por unidad de área, corrigen esa dependencia.
- Reducir la imperfección del elemento débil al 1 % y repetir la advertencia. ¿Qué paso hace falta ahora?
- Controlar con el control indirecto el desplazamiento del extremo en vez del alargamiento del elemento débil. ¿Hasta dónde llega en la barra larga?

Referencias. R. de Borst, "Computation of post-bifurcation and post-failure behavior of strain-softening solids", *Computers & Structures* 25 (1987) 211-224. M. A. Crisfield, *Non-linear Finite Element Analysis of Solids and Structures*, vol. 1, Wiley, 1991, cap. 9. Z. P. Bažant y J. Planas, *Fracture and Size Effect in Concrete and Other Quasibrittle Materials*, CRC Press, 1998.

Archivos del ejemplo. La carpeta `examples/barra_dano_localizado/` contiene el script `run.py`, los resultados en `resultados.json` y las dos figuras.

Solidum FEM — Manual de Ejemplos

Sigla del manual: FF-ME

Compilado el 23 de septiembre de 2026, 19:18

Commit Git: b93034d (con cambios sin confirmar)

Documento generado automáticamente desde `manuals/sources/examples/` y los scripts de `examples/` mediante `python manuals/build_example_manual.py`. No editar directamente el PDF: cualquier corrección debe realizarse sobre el archivo fuente correspondiente y regenerar el manual.